

Connect Chat API Documentation

This document describes the current REST and WebSocket APIs for building a mobile client.

Client-facing service URLs depend on the environment.

For native k3s on dev0 using direct Kubernetes NodePorts:

```
Service | URL
Identity Service | http://dev0:30081
Group Service | http://dev0:30082
Chat Service WebSocket | ws://dev0:30083/ws/chat
```

For native k3s on dev0 with the optional Nginx reverse proxy from [.docs/dev0-linux-k3s-setup.md](#):

```
Service | URL
Identity Service | http://dev0:8081
Group Service | http://dev0:8082
Chat Service WebSocket | ws://dev0:8083/ws/chat
```

For local machine development with Docker Compose services or k3d port mappings:

```
Service | URL
Identity Service | http://localhost:8081
Group Service | http://localhost:8082
Chat Service WebSocket | ws://localhost:8083/ws/chat
Message Storage Service | http://localhost:8084 if running locally or port-forwarded
Presence Service | http://localhost:8085 if running locally or port-forwarded
```

Examples in this document use variables so the same commands work in each environment:

```
# Native k3s on dev0, direct NodePorts
export IDENTITY_BASE_URL="http://dev0:30081"
export GROUP_BASE_URL="http://dev0:30082"
export CHAT_HTTP_BASE_URL="http://dev0:30083"
export CHAT_WS_URL="ws://dev0:30083/ws/chat"

# Native k3s on dev0, optional Nginx proxy
# export IDENTITY_BASE_URL="http://dev0:8081"
# export GROUP_BASE_URL="http://dev0:8082"
# export CHAT_HTTP_BASE_URL="http://dev0:8083"
# export CHAT_WS_URL="ws://dev0:8083/ws/chat"

# Local development
# export IDENTITY_BASE_URL="http://localhost:8081"
# export GROUP_BASE_URL="http://localhost:8082"
# export CHAT_HTTP_BASE_URL="http://localhost:8083"
# export CHAT_WS_URL="ws://localhost:8083/ws/chat"
```

Common Conventions

All REST services that return JSON use this envelope:

```
{
  "metadata": {
    "requestId": "request/session id",
```

```
    "timestamp": "2026-05-19T13:25:19.952135Z",
    "status": 200,
    "message": "Human-readable result"
  },
  "data": {},
  "error": null
}
```

Error response shape:

```
{
  "metadata": {
    "requestId": "request/session id",
    "timestamp": "2026-05-19T13:25:19.952135Z",
    "status": 400,
    "message": "Bad Request"
  },
  "data": null,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed"
  }
}
```

Use ISO-8601 timestamps. UUID fields are strings. Phone numbers use E.164 format, for example +15551234567.

Authenticated user APIs require:

Authorization: Bearer <user-access-token>

Internal APIs require:

Authorization: Bearer <internal-service-token>

Mobile clients should use only user access tokens. Internal service tokens are for backend services only.

Identity Service

Base URL:

`$IDENTITY_BASE_URL/api/v1/identity`

Register

Starts registration and sends a verification code by SMS.

POST /api/v1/identity/auth/register
Content-Type: application/json

Request:

```
{
  "phoneNumber": "+15551234567",
  "firstName": "Dami",
  "lastName": "Begud",
  "nickname": "dami",
  "dateOfBirth": "1995-04-20",
  "country": "US"
}
```

Validation:

Field	Required	Rule
phoneNumber	yes	E.164 format, example +15551234567
firstName	yes	max 100 chars
lastName	yes	max 100 chars
nickname	no	max 100 chars
dateOfBirth	yes	date in the past, format YYYY-MM-DD
country	yes	max 100 chars

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Registration verification code sent"
  },
  "data": null,
  "error": null
}
```

Example:

```
curl -X POST "$IDENTITY_BASE_URL/api/v1/identity/auth/register" \
-H "Content-Type: application/json" \
-d '{
  "phoneNumber": "+15551234567",
  "firstName": "Dami",
  "lastName": "Begud",
  "nickname": "dami",
  "dateOfBirth": "1995-04-20",
  "country": "US"
}'
```

Verify Registration

Verifies the SMS code and returns user tokens.

POST /api/v1/identity/auth/register/verify
Content-Type: application/json

Request:

```
{
  "phoneNumber": "+15551234567",
  "verificationCode": "123456"
}
```

Validation:

Field	Required	Rule
phoneNumber	yes	E.164 format
verificationCode	yes	exactly 6 digits

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Registration verified"
  },
  "data": {
    "accessToken": "jwt-access-token",
    "refreshToken": "uuid-refresh-token"
  },
  "error": null
}
```

Example:

```
curl -X POST "$IDENTITY_BASE_URL/api/v1/identity/auth/register/verify" \
  -H "Content-Type: application/json" \
  -d '{
    "phoneNumber": "+15551234567",
    "verificationCode": "123456"
  }'
```

Refresh Token

Returns a new access token and refresh token.

POST /api/v1/identity/auth/token/refresh
Content-Type: application/json

Request:

```
{
  "refreshToken": "uuid-refresh-token"
}
```

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Token refreshed"
  },
  "data": {
    "accessToken": "new-jwt-access-token",
    "refreshToken": "new-uuid-refresh-token"
  },
  "error": null
}
```

Validate Token

Validates a bearer token and returns token identity. This is mainly for backend services.

POST /api/v1/identity/auth/token/validate

Authorization: Bearer <token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Token is valid"
  },
  "data": {
    "subject": "user-id-or-service-id",
    "tokenType": "user",
    "role": "USER",
    "expiresAt": "2026-05-19T18:30:00Z"
  },
  "error": null
}
```

User access tokens contain:

```
{
  "sub": "user-uuid",
  "token_type": "user",
  "role": "USER"
}
```

The mobile app can decode the access token locally to read the current user id from sub, but authorization should still rely on backend validation.

Issue Internal Service Token

Internal only. Mobile clients should not call this endpoint.

POST /api/v1/identity/auth/service-token

Content-Type: application/json

Request:

```
{
  "clientId": "chat-service",
  "clientSecret": "local-chat-service-secret"
}
```

Response:

```
{
  "metadata": {
    "status": 200,
```

```
    "message": "Service token issued"
  },
  "data": {
    "accessToken": "jwt-service-token",
    "tokenType": "service",
    "role": "INTERNAL_SERVICE",
    "expiresAt": "2026-05-19T18:30:00Z"
  },
  "error": null
}
```

Get User By ID

Internal only. Mobile clients should not call this endpoint.

GET /api/v1/identity/users/{userId}
Authorization: Bearer <internal-service-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "User fetched"
  },
  "data": {
    "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
    "phoneNumber": "+15551234567"
  },
  "error": null
}
```

Get User By Phone Number

Internal only. Mobile clients should not call this endpoint. Chat-service uses it to resolve private message recipients before enqueueing messages.

GET /api/v1/identity/users/by-phone/{phoneNumber}
Authorization: Bearer <internal-service-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "User fetched"
  },
  "data": {
    "userId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
    "phoneNumber": "+15557654321"
  },
  "error": null
}
```

Identity Smoke Check

GET /api/v1/identity

Returns 200 OK with empty body.

Group Service

Base URL:

`$GROUP_BASE_URL/api/v1/groups`

All group APIs require a user access token unless explicitly called by an internal service.

Create Group

POST /api/v1/groups

Authorization: Bearer <user-access-token>

Content-Type: application/json

Request:

```
{
  "name": "Best Friends"
}
```

Validation:

Field	Required	Rule
name	yes	not blank, max 120 chars

Response:

```
{
  "metadata": {
    "status": 201,
    "message": "Group created"
  },
  "data": {
    "id": "9d46599f-27cb-4d2b-8d3b-4ffffce6773d3",
    "ownerId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
    "name": "Best Friends",
    "createdAt": "2026-05-19T13:25:19.952135Z"
  },
  "error": null
}
```

Example:

```
curl -X POST "$GROUP_BASE_URL/api/v1/groups" \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "name": "Best Friends" }'
```

Add Group Member

Only the group owner can add members.

POST /api/v1/groups/{groupId}/members
Authorization: Bearer <owner-access-token>
Content-Type: application/json

Request:

```
{
  "phoneNumber": "+15557654321"
}
```

Group-service resolves phoneNumber to the member's internal UUID through identity-service before storing the group membership.

Response:

```
{
  "metadata": {
    "status": 201,
    "message": "Group member added"
  },
  "data": {
    "groupId": "9d46599f-27cb-4d2b-8d3b-4ffffce6773d3",
    "userId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
    "role": "MEMBER",
    "joinedAt": "2026-05-19T13:25:19.952135Z"
  },
  "error": null
}
```

Remove Group Member

Only the group owner can remove another member. The owner cannot remove themselves.

DELETE /api/v1/groups/{groupId}/members/{userId}
Authorization: Bearer <owner-access-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Group member removed"
  },
  "data": null,
  "error": null
}
```

Leave Group

A member can remove themselves from a group. The owner cannot leave through this endpoint until ownership transfer exists.

DELETE /api/v1/groups/{groupId}/members/me
Authorization: Bearer <user-access-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Left group"
  },
  "data": null,
  "error": null
}
```

Get Group Members

Available to group members and internal services.

GET /api/v1/groups/{groupId}/members
Authorization: Bearer <user-access-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Group members fetched"
  },
  "data": [
    {
      "groupId": "9d46599f-27cb-4d2b-8d3b-4ffffce6773d3",
      "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
      "role": "OWNER",
      "joinedAt": "2026-05-19T13:25:19.952135Z"
    },
    {
      "groupId": "9d46599f-27cb-4d2b-8d3b-4ffffce6773d3",
      "userId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
      "role": "MEMBER",
      "joinedAt": "2026-05-19T13:26:00.000000Z"
    }
  ],
  "error": null
}
```

Chat WebSocket API

Chat uses raw WebSocket with STOMP.

Endpoint:

\$CHAT_WS_URL

STOMP app destination prefix:

/app

STOMP user destination prefix:

/user

Connect with user access token:

```
CONNECT
Authorization: Bearer <user-access-token>
accept-version:1.2,1.1,1.0
heart-beat:10000,10000
```

Chat-service validates this token through identity-service during CONNECT. The token must be a user token with role USER.

Required Subscriptions

Subscribe to private messages:

/user/queue/private-messages

Payload received:

```
{
  "messageId": "42fa8a65-a118-49c9-bd50-0ff96116d0e8",
  "senderId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
  "senderPhoneNumber": "+15551234567",
  "recipientId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
  "content": "Hello World",
  "sentAt": "2026-05-19T13:30:00.000000Z"
}
```

Subscribe to private message status updates:

/user/queue/private-message-status

Payload received:

```
{
  "messageId": "42fa8a65-a118-49c9-bd50-0ff96116d0e8",
  "senderId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
  "recipientId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
  "status": "DELIVERED",
  "actorUserId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
  "occurredAt": "2026-05-19T13:30:05.000000Z"
}
```

Status values:

Status | Meaning

SENT | Message was stored successfully by message-storage-service.

DELIVERED | Recipient acknowledged receiving the message.

READ | Recipient acknowledged reading the message.

Send Private Message

Destination:

/app/chat.private

Body:

```
{
  "recipientPhoneNumber": "+15557654321",
  "content": "Hello World"
}
```

Validation:

Field | Required | Rule

recipientPhoneNumber | yes | E.164 format

content | yes | not blank, max 4000 chars

Notes:

- The sender is resolved from the WebSocket access token.
- Chat-service resolves recipientPhoneNumber to the recipient's internal UUID through identity-service before enqueueing the message.
- The private message payload includes senderPhoneNumber so receivers can display the sender's phone number.
- The message is eventually delivered to both sender and recipient on /user/queue/private-messages.
- The message is also stored asynchronously by message-storage-service.

Acknowledge Delivered

Only the recipient can acknowledge DELIVERED.

Destination:

/app/chat.private.delivered

Body:

```
{
  "messageId": "42fa8a65-a118-49c9-bd50-0ff96116d0e8"
}
```

Acknowledge Read

Only the recipient can acknowledge READ.

Destination:

```
/app/chat.private.read
```

Body:

```
{
  "messageId": "42fa8a65-a118-49c9-bd50-0ff96116d0e8"
}
```

STOMP JavaScript Example

This is a minimal browser-compatible example. Mobile clients should use the equivalent STOMP client for their platform.

```
import { Client } from "@stomp/stompjs";

const accessToken = "user-access-token";
// Use the same value as CHAT_WS_URL for your environment.
const chatWsUrl = "ws://dev0:30083/ws/chat";

const client = new Client({
  brokerURL: chatWsUrl,
  connectHeaders: {
    Authorization: `Bearer ${accessToken}`,
  },
  onConnect: () => {
    client.subscribe("/user/queue/private-messages", (frame) => {
      const message = JSON.parse(frame.body);
      console.log("message", message);

      client.publish({
        destination: "/app/chat.private.delivered",
        body: JSON.stringify({ messageId: message.messageId }),
        headers: { "content-type": "application/json" },
      });
    });

    client.subscribe("/user/queue/private-message-status", (frame) => {
      console.log("status", JSON.parse(frame.body));
    });
  },
});

client.activate();

function sendMessage(recipientPhoneNumber, content) {
  client.publish({
    destination: "/app/chat.private",
    body: JSON.stringify({ recipientPhoneNumber, content }),
    headers: { "content-type": "application/json" },
  });
}

function markRead(messageId) {
  client.publish({
```

```
    destination: "/app/chat.private.read",
    body: JSON.stringify({ messageId }),
    headers: { "content-type": "application/json" },
  });
}
```

Presence Service

Presence-service is internal only. Mobile clients should not call it directly. Chat-service calls it with an internal service token.

Base URL:

`http://presence-service:8085/api/v1/presence`

When debugging from outside the cluster, port-forward it first:

```
kubectll -n connect-chat port-forward svc/presence-service 8085:8085
```

Then use:

`http://localhost:8085/api/v1/presence`

Register Session

Internal only.

```
POST /api/v1/presence/sessions
Authorization: Bearer <internal-service-token>
Content-Type: application/json
```

Request:

```
{
  "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
  "sessionId": "websocket-session-id",
  "instanceId": "chat-service:local-chat-service-1"
}
```

Response:

```
{
  "metadata": {
    "status": 201,
    "message": "Presence session registered"
  },
  "data": null,
  "error": null
}
```

Remove Session

Internal only.

```
DELETE /api/v1/presence/sessions/{sessionId}
Authorization: Bearer <internal-service-token>
```

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Presence session removed"
  },
  "data": null,
  "error": null
}
```

Get User Presence

Internal only.

```
GET /api/v1/presence/users/{userId}
Authorization: Bearer <internal-service-token>
```

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Presence fetched"
  },
  "data": {
    "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
    "status": "ONLINE",
    "sessions": [
      {
        "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
        "sessionId": "websocket-session-id",
        "instanceId": "chat-service:local-chat-service-1",
        "connectedAt": "2026-05-19T13:30:00.000000Z",
        "lastHeartbeatAt": "2026-05-19T13:30:00.000000Z"
      }
    ],
    "updatedAt": "2026-05-19T13:30:00.000000Z"
  },
  "error": null
}
```

Presence statuses:

ONLINE
OFFLINE

Current implementation does not have heartbeat refresh yet. lastHeartbeatAt is set when the session is registered.

Check If User Is Online

Internal only.

GET /api/v1/presence/users/{userId}/online
Authorization: Bearer <internal-service-token>

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Presence status fetched"
  },
  "data": {
    "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
    "online": true
  },
  "error": null
}
```

Batch Presence Lookup

Internal only.

POST /api/v1/presence/users/lookup
Authorization: Bearer <internal-service-token>
Content-Type: application/json

Request:

```
{
  "userIds": [
    "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
    "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09"
  ]
}
```

Response:

```
{
  "metadata": {
    "status": 200,
    "message": "Presence lookup fetched"
  },
  "data": [
    {
      "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
      "status": "ONLINE",
      "sessions": [
        {
          "userId": "793de6b4-7ced-4a80-80c7-dd22d9b90a72",
          "sessionId": "websocket-session-id",
          "instanceId": "chat-service:local-chat-service-1",
          "connectedAt": "2026-05-19T13:30:00.000000Z",

```

```

        "lastHeartbeatAt": "2026-05-19T13:30:00.000000Z"
    }
],
"updatedAt": "2026-05-19T13:30:00.000000Z"
},
{
    "userId": "ac9b3a0a-bedb-45bc-975c-9a3b83a6ca09",
    "status": "OFFLINE",
    "sessions": [],
    "updatedAt": "2026-05-19T13:31:00.000000Z"
}
],
"error": null
}

```

Message Storage Service

Message-storage-service currently has no mobile-facing REST API.

It consumes RabbitMQ events from chat-service and stores private messages/status updates. Mobile clients receive messages and statuses through chat-service WebSocket subscriptions.

Health Endpoints

Each service exposes health:

GET /actuator/health

Examples:

```

curl "$IDENTITY_BASE_URL/actuator/health"
curl "$GROUP_BASE_URL/actuator/health"

```

For chat health:

```

curl "$CHAT_HTTP_BASE_URL/actuator/health"

```

For internal services, port-forward before checking health:

```

kubectl -n connect-chat port-forward svc/message-storage-service 8084:8084
kubectl -n connect-chat port-forward svc/presence-service 8085:8085

```

Then, in separate terminals:

```

curl "http://localhost:8084/actuator/health"
curl "http://localhost:8085/actuator/health"

```

Recommended Mobile App Flow

1. Register user with POST /identity/auth/register.

2. Verify registration with POST /identity/auth/register/verify.
3. Store accessToken and refreshToken securely.
4. Use accessToken for group REST APIs.
5. Connect to chat WebSocket with Authorization: Bearer <accessToken>.
6. Subscribe to /user/queue/private-messages.
7. Subscribe to /user/queue/private-message-status.
8. Send private messages to /app/chat.private with recipientPhoneNumber.
9. When receiving a message as recipient, publish delivered/read acknowledgements.
10. Refresh tokens with POST /identity/auth/token/refresh before or after access token expiration.